



PERTEMUAN 05

Analisis Intensitas dan Histogram Citra

(Histogram, Ekualisasi & Dithering)

Pengolahan Citra dan Visi Komputer · RTI235007



Overview



Histogram citra

Membaca sebaran intensitas dan menilai kualitas sebuah citra



Histogram equalization

Meratakan sebaran intensitas agar kontras naik



Dithering

Menjaga kesan gradasi saat jumlah warna dipangkas



01

Histogram citra

Sebaran intensitas dan cara membacanya



Apa itu histogram citra

Citra → hitung frekuensi tiap nilai intensitas → grafik sebaran

SUMBU X dan Y pada Histogram

- Sumbu x: nilai intensitas piksel, 0 sampai $L-1$
- Sumbu y: jumlah piksel yang punya nilai tersebut
- Kiri = area gelap, tengah = abu-abu, kanan = area terang
- Grayscale menghasilkan satu histogram

Citra warna menghasilkan satu histogram untuk tiap kanal

YANG BISA DINILAI DARI HISTOGRAM

- Citra terlalu gelap atau terlalu terang
- Kontras lebar atau sempit
- Jumlah kelompok objek yang berbeda terang gelapnya

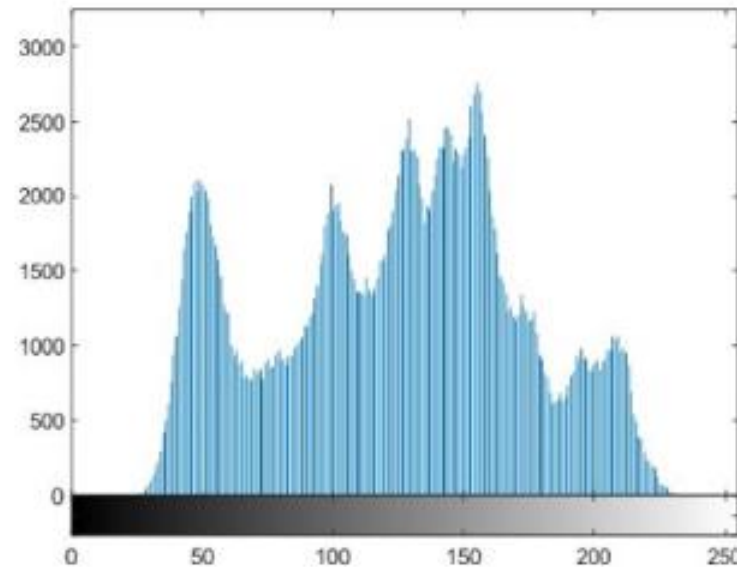
Histogram tidak menyimpan posisi piksel, hanya jumlahnya

Dua citra yang sama sekali berbeda bisa memiliki histogram yang sama persis.

Contoh histogram citra grayscale



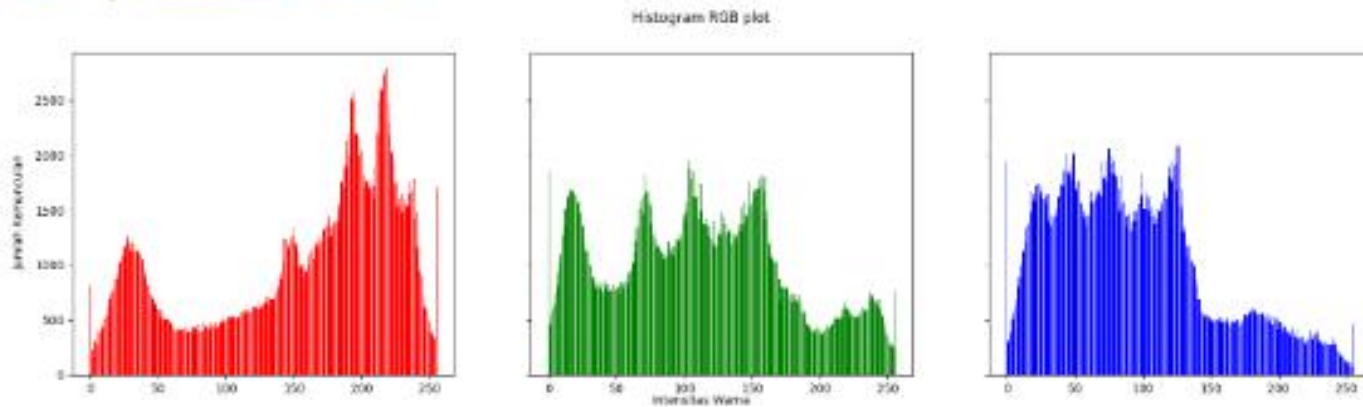
(a) Citra Grayscale



(b) Histogram Citra

- Sumbu x: nilai intensitas 0 sampai 255
- Sumbu y: jumlah piksel pada nilai tersebut
- Puncak di tengah menandakan sebagian besar piksel bernilai menengah

Histogram citra berwarna



- SATU HISTOGRAM TIAP KANAL
- Citra warna menghasilkan tiga histogram: merah, hijau, dan biru
- Bentuk ketiganya bisa berbeda jauh

Pada citra grayscale ketiganya identik

Menghitung histogram

$h(j)$ = jumlah piksel yang bernilai j , untuk $j = 0, 1, \dots, L-1$

LANGKAH

- Siapkan array dengan panjang L , isi seluruhnya dengan 0
- Telusuri setiap piksel citra satu per satu
- Tambah 1 pada array sesuai nilai piksel tersebut
- Untuk citra warna, ulangi untuk kanal R, G, dan B

Jumlah seluruh isi larik harus sama dengan $M \times N$

NORMALISASI

- $p(j) = h(j) / (M \times N)$, yaitu peluang kemunculan tiap nilai
- Nilainya berada di rentang 0 sampai 1
- Jumlah seluruh $p(j)$ selalu sama dengan 1
- Bentuk grafiknya tidak berubah, hanya skalanya yang berbeda

Normalisasi inilah yang dipakai sebagai dasar Histogram Equalization

Histogram ternormalisasi membuat dua citra dengan ukuran berbeda bisa dibandingkan.

Flowchart membuat histogram citra



ALUR PROGRAM

- Baca citra masukan
- Hitung jumlah kemunculan setiap nilai warna pada seluruh piksel
- Buat grafik dengan sumbu x nilai intensitas dan sumbu y jumlah kemunculan

Output: histogram citra

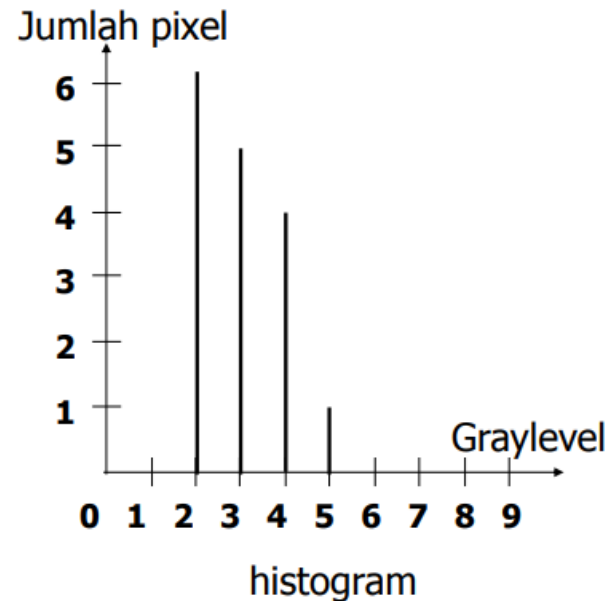
Menghitung histogram citra 4 x 4

Sumber: ALI JAVED, Digital Image Processing, Chapter # 3,
Image Enhancement in Spatial Domain

Contoh:

2	3	3	2
4	2	4	3
3	2	3	5
2	4	2	4

4x4 image
Grayscale = [0,9]



- Misalkan sebuah citra mempunyai L level nilai keabuan, $[0, L-1]$.
- Hitung frekuensi kemunculan setiap nilai keabuan j dengan cara menghitung jumlah pixel yang mempunyai nilai keabuan tersebut.
- Perhitungan ini dilakukan untuk $j = 0, 1, 2, \dots, L-1$

Tinggi batang pada grafik adalah jumlah kemunculan itu



Enam bentuk histogram yang sering ditemui

1

Gelap

Menumpuk di kiri

2

Terang

Menumpuk di kanan

3

Rendah

Kontras sempit

4

Tinggi

Kontras lebar

5

Bimodal

Dua puncak

6

Clipping

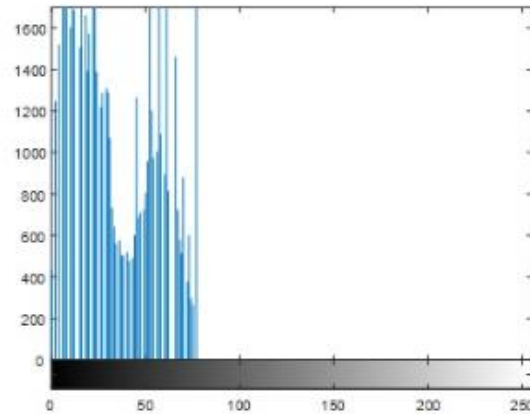
Menumpuk di 0/255

Histogram citra gelap dan citra terang



(a) Citra gelap

Citra gelap: batang menumpuk di sebelah kiri
(memiliki banyak piksel dengan nilai intensitas mendekati 0)

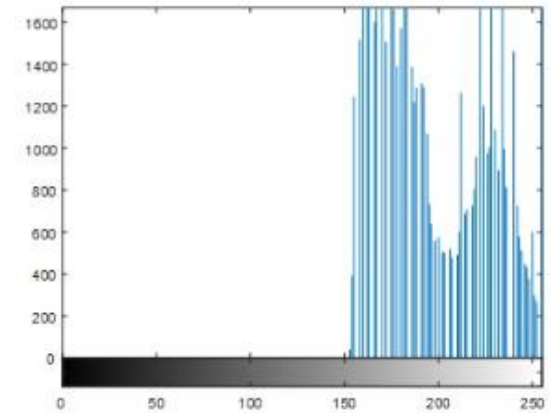


(b) Histogram citra gelap



(a) Citra terang

Citra terang: batang menumpuk di sebelah kanan
(memiliki banyak piksel dengan nilai intensitas mendekati 255)

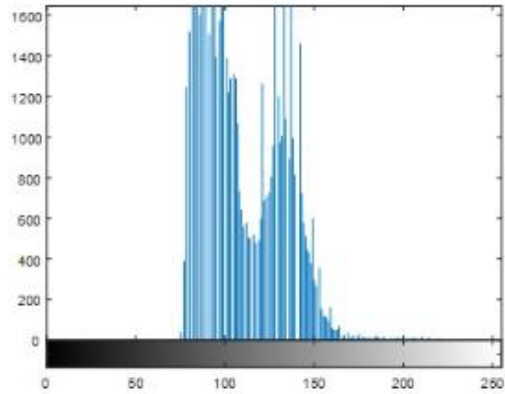


(b) Histogram citra terang

Histogram kontras rendah dan kontras tinggi



(a) Citra dengan kontras rendah

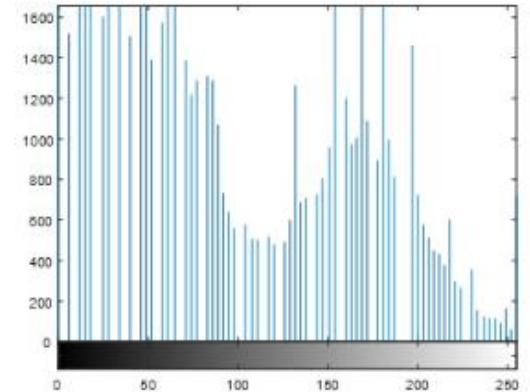


(b) Histogram citra dengan kontras rendah

Kontras rendah: hanya mengisi sebagian kecil rentang (memiliki range nilai intensitas yang sempit)



(a) Citra dengan kontras tinggi



(b) Histogram citra dengan kontras tinggi

Kontras tinggi: mengisi hampir seluruh rentang (memiliki range nilai intensitas yang lebar)

Latihan 1 • histogram dan normalisasi citra 4 x 4

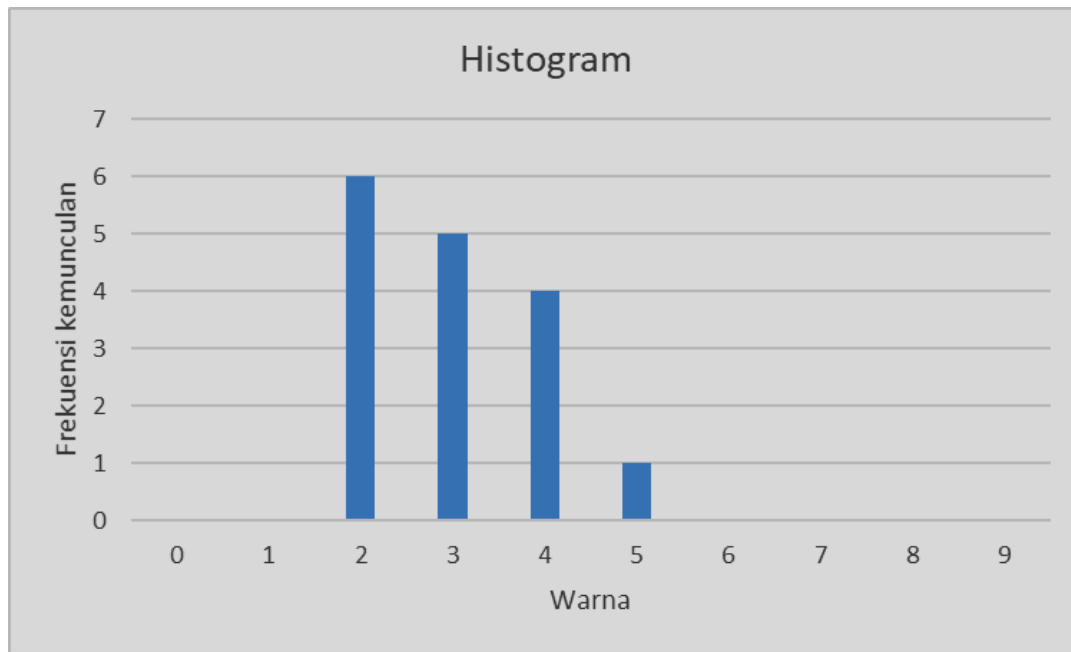
Citra Grayscale 9 warna

2	3	3	2
4	2	4	3
3	2	3	5
2	4	2	4

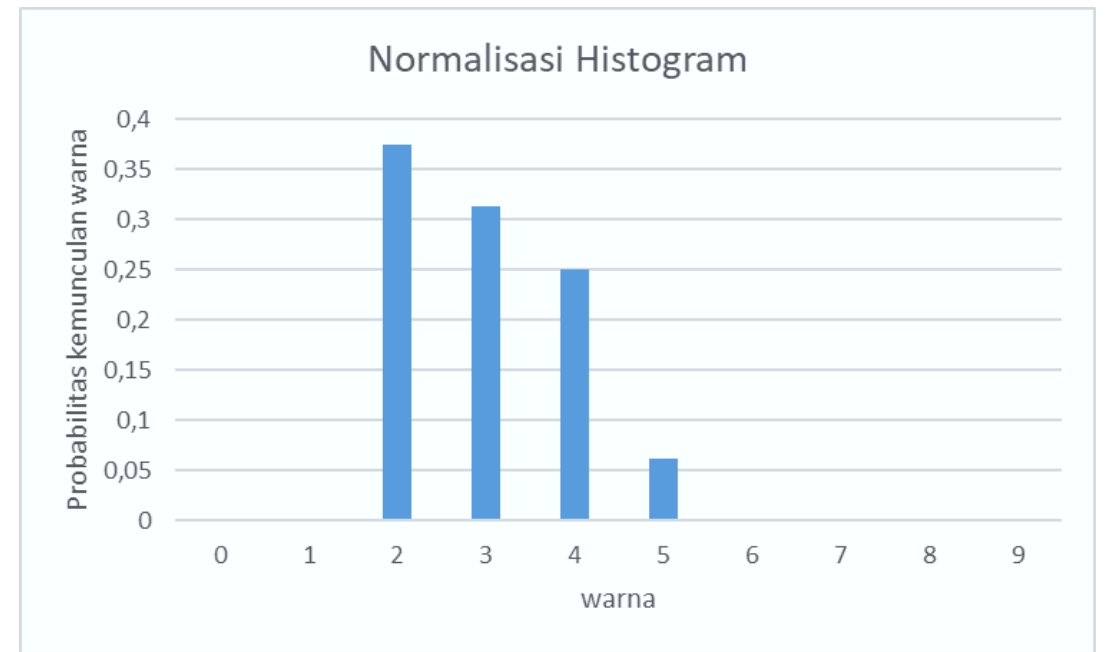
Nilai	0	1	2	3	4	5	6	7	8
Frekuensi	0	0	6	5	4	1	0	0	0
$p(j)$	0	0	0,375	0,3125	0,25	0,0625	0	0	0
CDF (Cumulative Distribution Function)	0	0	0,375	0,6875	0,9375	1	1	1	1

Citra 4 x 4 dengan 9 Warna → Total piksel 16, sehingga $p(2) = 6/16 = 0,375$.

Histogram dan histogram ternormalisasi



Sumbu y: frekuensi kemunculan warna



Sumbu y: probabilitas, jumlah = 1



02

Histogram equalization

Meratakan sebaran histogram untuk meningkatkan kontras



Histogram Equalization

Histogram menumpuk → fungsi transformasi $T(r)$ → sebaran lebih merata

IDE DASAR

Derajat keabuan lama r dipetakan ke derajat keabuan baru s

- Pemetaan diambil dari distribusi kumulatif histogram citra itu sendiri
- Nilai yang jarang muncul dirapatkan, yang sering muncul diregangkan
- Karena itu kontras naik tanpa perlu menentukan parameter apa pun

YANG PERLU DIINGAT

Hasilnya tidak pernah benar-benar rata pada citra digital

- Beberapa level lama bisa jatuh ke level baru yang sama
- Operasi ini global: satu pemetaan untuk seluruh citra
- Noise pada daerah gelap ikut dikuatkan

Termasuk operasi global, bukan point operation biasa

Histogram Equalization memakai statistik citra itu sendiri, jadi hasilnya berbeda untuk setiap citra.

$$s_k = (L - 1) \sum_{i=0}^k p_i$$

s_k = warna baru dari warna k

L = Jumlah variasi warna pada citra

$k = 0, 1, 2, 3 \dots (L - 1)$

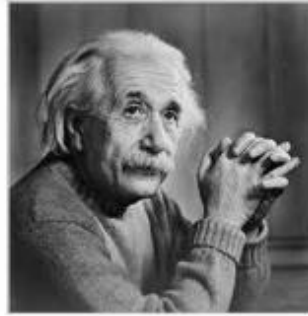
p_i = probabilitas warna i

Sebelum dan sesudah Histogram Equalization

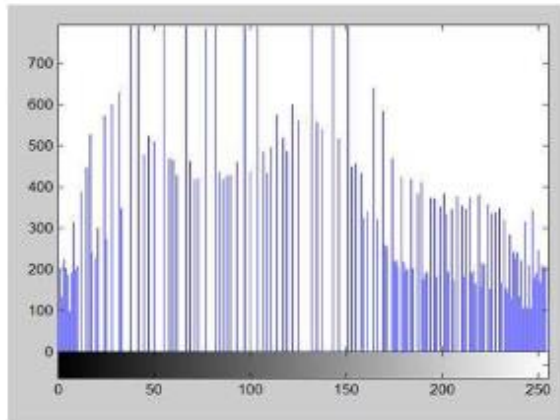
New Image



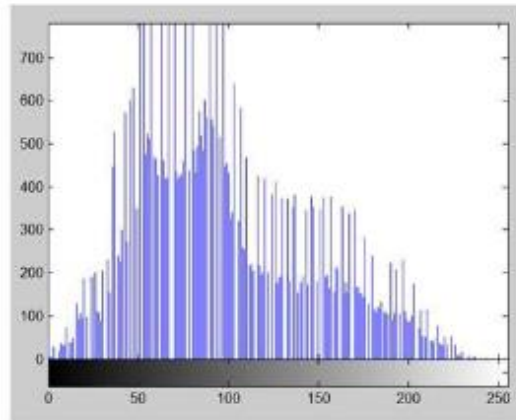
Old image



New Histogram



Old Histogram



- Citra lama: histogram menumpuk pada rentang sempit
- Citra baru: batang tersebar hampir ke seluruh rentang
- Detail pada daerah gelap menjadi lebih terlihat

Perhatikan munculnya celah kosong pada histogram baru

Fungsi transformasi

$$S = T(r) = (L - 1) * CDF(r)$$

DESKRIPSI

- r : derajat keabuan lama, 0 sampai $L-1$
- s : derajat keabuan baru hasil pemetaan
- L : jumlah level intensitas, 256 untuk citra 8 bit
- $CDF(r)$: **Cumulative Distribution Function** jumlah kumulatif $p(0)$ sampai $p(r)$
- $CDF(k) = \sum p(j)$ untuk $j = 0 \dots k$.

$M \times N$: jumlah seluruh piksel citra

CDF selalu naik, jadi urutan gelap ke terang tidak pernah terbalik setelah Histogram Equalization .



Lima langkah Histogram Equalization

1 Histogram
Hitung $h(r)$

2 Normalisasi
 $p(r) = h(r)/MN$

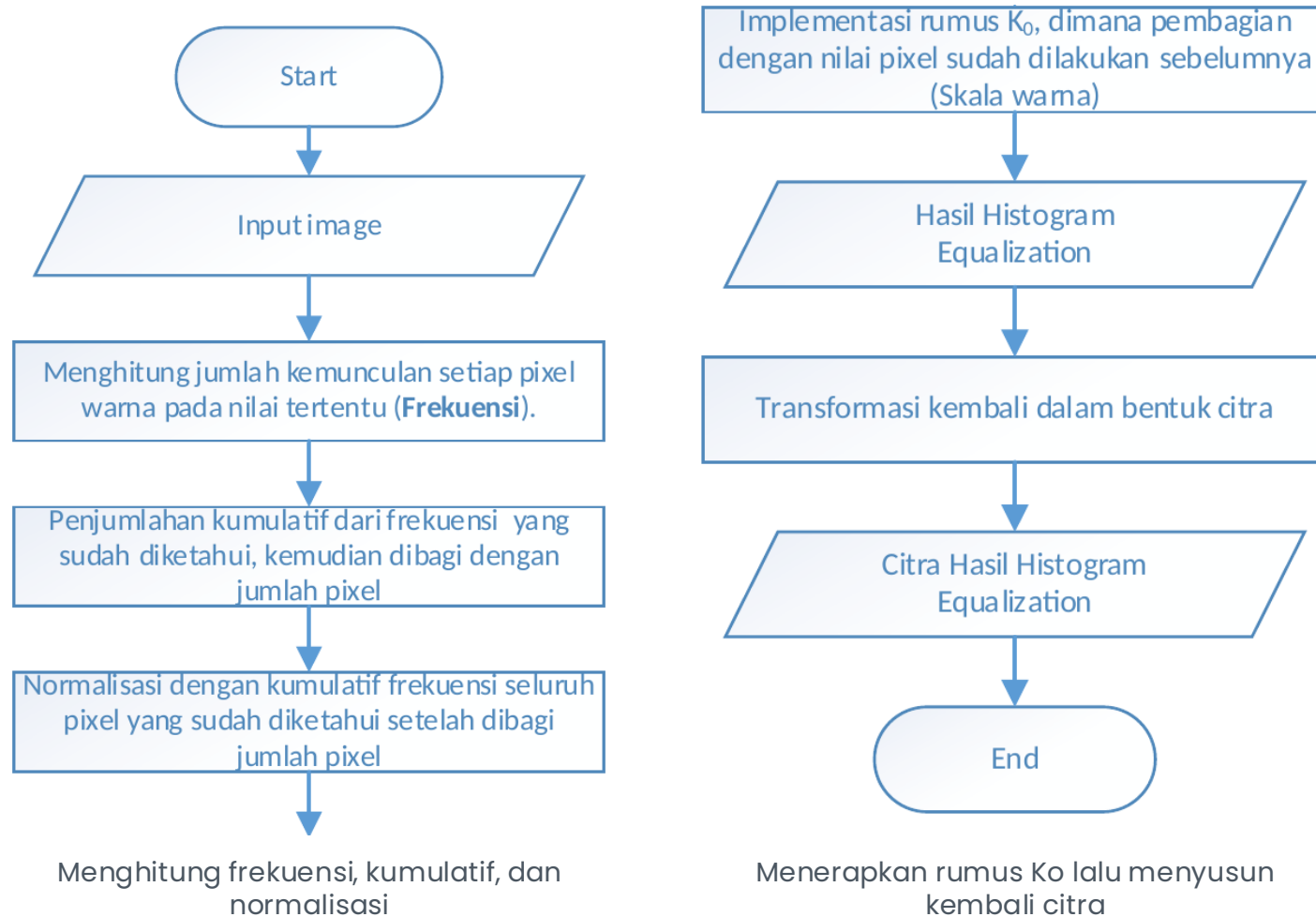
3 Kumulatif
 $CDF = \text{jumlah } p(0..r)$

4 Skala
 $s = (L-1) \times CDF$

5 Pembulatan
Bulatkan ke integer

6 Pemetaan
Ganti nilai r menjadi s

Flowchart Histogram Equalization



Contoh Perhitungan citra 3 bit ($L = 8$, $MN = 4096$)

rk	nk	p(rk)	CDF	s = 7 x CDF	s dibulatkan
r0 = 0	790	0,19	0,19	1,33	1
r1 = 1	1023	0,25	0,44	3,08	3
r2 = 2	850	0,21	0,65	4,55	5
r3 = 3	656	0,16	0,81	5,67	6
r4 = 4	329	0,08	0,89	6,23	6
r5 = 5	245	0,06	0,95	6,65	7
r6 = 6	122	0,03	0,98	6,86	7
r7 = 7	81	0,02	1,00	7,00	7

Delapan level lama menyusut menjadi enam level baru: 1, 3, 5, 6, 7 → Penyebab hasil tidak pernah benar-benar rata.

Ketika Histogram Equalization global tidak menolong

Satu pemetaan untuk seluruh citra → daerah terang dan gelap diperlakukan sama

Ciri-ciri

- Citra backlight: latar terang ikut naik, objek tetap gelap
- Noise pada bayangan ikut menonjol dan terlihat berbintik
- Warna kulit dan langit bisa terlihat keras dan tidak wajar
- Daerah rata yang luas berubah jadi bercak

CLAHE SEBAGAI JALAN KELUAR

Citra dipotong menjadi petak kecil, tiap petak diekualisasi sendiri

- clipLimit membatasi tinggi histogram agar noise tidak meledak
- tileGridSize menentukan ukuran petak
- Batas antar petak dihaluskan dengan interpolasi bilinear

`cv.createCLAHE(clipLimit=..., tileGridSize=(..., ...))`

Untuk foto KTM di ruangan gelap, CLAHE hampir selalu lebih terbaca daripada Histogram Equalization

Histogram Equalization pada citra berwarna

Jangan ekualisasi R, G, dan B secara terpisah

EKUALISASI TIAP KANAL RGB

Tiap kanal mendapat pemetaan yang berbeda

- Perbandingan antar kanal berubah
- Akibatnya warna bergeser, sering tampak kebiruan atau kemerahan
- Hampir tidak pernah dipakai pada praktik

EKUALISASI KANAL TERANG SAJA

- Ubah ke YCrCb, ekualisasi kanal Y, lalu kembalikan ke BGR
- Bisa juga kanal V pada HSV atau kanal L pada LAB
- Kecerahan naik, informasi warna tetap utuh
- Inilah cara yang dipakai pada pengolahan foto

Aturan praktisnya: perbaiki kecerahan pada kanal terang, jangan pada kanal warna.

Latihan 2

citra 4 x 4 dari Latihan 1 (L = 9)

Warna lama	0	1	2	3	4	5	6	7	8
CDF	0	0	0,375	0,6875	0,9375	1	1	1	1
s = 8 x CDF	0	0	3	5,5	7,5	8	8	8	8
Warna baru	0	0	3	6	8	8	8	8	8

Piksel bernilai 2 menjadi 3, bernilai 3 menjadi 6, bernilai 4 dan 5 sama-sama menjadi 8. Sebaran melebar, tetapi dua level lama menyatu pada satu level baru.

Detail Perhitungan Latihan 2

Warna	Probabilita s	Warna Baru
0	0	0
1	0	0
2	0,375	3
3	0,3125	6
4	0,25	8
5	0,0625	8
6	0	8
7	0	8
8	0	8

$$s_k = (L - 1) \sum_{i=0}^k p_i$$

$k = 0$

$$s_0 = (9 - 1) \sum_{i=0}^0 p_i$$
$$= 8x0$$

$k = 1$

$$s_1 = (9 - 1) \sum_{i=0}^1 p_i$$
$$= 8x(p_0 + p_1)$$
$$= 8x(0 + 0)$$
$$= 0$$

$k = 2$

$$s_2 = (9 - 1) \sum_{i=0}^2 p_i$$
$$= 8x(p_0 + p_1 + p_2)$$
$$= 8x(0 + 0 + 0,375)$$
$$= 2.8$$

$k = 3$

$$s_3 = (9 - 1) \sum_{i=0}^3 p_i$$
$$= 8x(p_0 + p_1 + p_2 + p_3)$$
$$= 8x(0 + 0 + 0,375 + 0,3125)$$
$$= 5,5$$

$k = 4$

$$s_4 = (9 - 1) \sum_{i=0}^4 p_i$$
$$= 8x(p_0 + p_1 + p_2 + p_3 + p_4)$$
$$= 8x(0 + 0 + 0,375 + 0,3125 + 0,25)$$
$$= 7,5$$

$k = 5$

$$s_5 = (9 - 1) \sum_{i=0}^5 p_i$$
$$= 8x(p_0 + p_1 + p_2 + p_3 + p_4 + p_5)$$
$$= 8x(0 + 0 + 0,375 + 0,3125 + 0,25 + 0,0625)$$
$$= 8$$

$k = 6$

$s_6 = 8$

$k = 7$

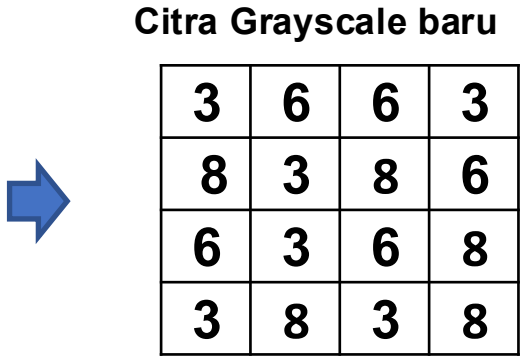
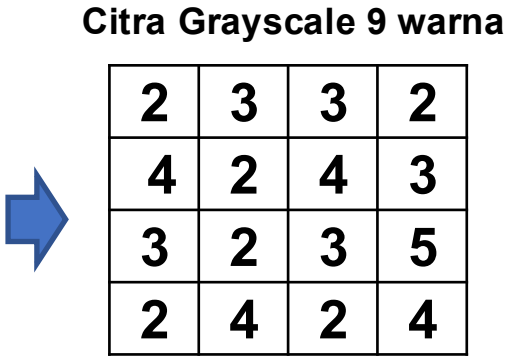
$s_6 = 8$

$k = 8$

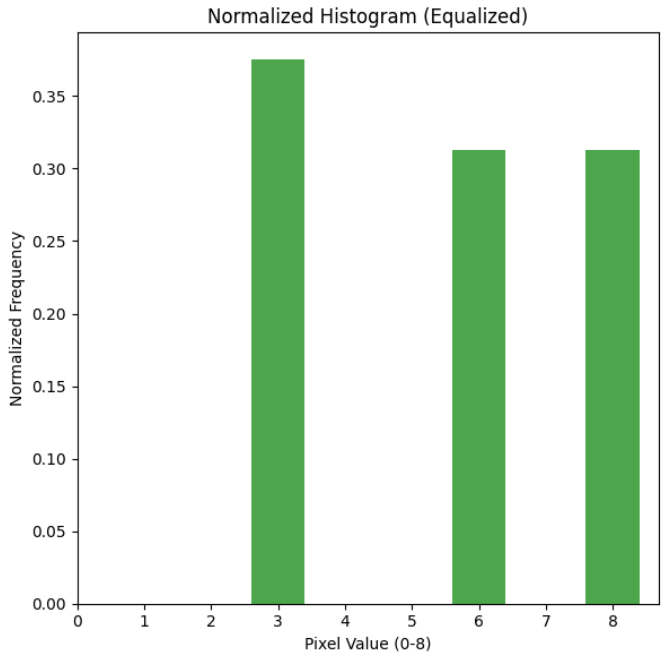
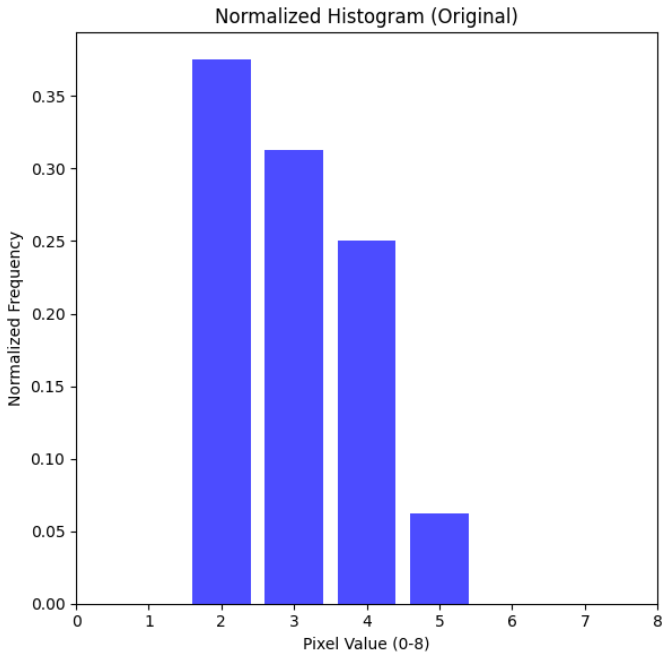
$s_6 = 8$

Detail Perhitungan Latihan 2

Warna	Probabilit	
	as	Baru
0	0	0
1	0	0
2	0,375	3
3	0,3125	6
4	0,25	8
5	0,0625	8
6	0	8
7	0	8
8	0	8



Warna	Frekuensi	Probabilitas	
0	0	0/16	0
1	0	0/16	0
2	0	0/16	0
3	6	6/16	0,375
4	0	0/16	0
5	0	0/16	0
6	5	5/16	0,3125
7	0	0/16	0
8	5	5/16	0,3125
9	0	0/16	0





03

Dithering & error diffusion

Menjaga kesan gradasi saat jumlah warna dipangkas



Mengapa jumlah warna perlu dipangkas?

Citra 24 bit seringkali harus ditampilkan dengan palet warna yang jauh lebih sedikit.

KAPAN KUANTISASI TERJADI ?

- Pencetakan: tinta hanya terbatas
- Berkas GIF dibatasi 256 warna
- Layar dengan kedalaman warna rendah
- Perangkat dengan memori kecil

AKIBAT PADA CITRA

- Gradasi terputus menjadi tangga warna
- Pita warna muncul di langit dan bayangan
- Perbedaan intensitas kecil hilang

Dithering menyebarkan error ini ke tetangganya.

Mata memadukan pola titik tersebut menjadi warna antara, walaupun dithering tidak menambah informasi apa pun.

Pengaruh pengurangan kedalaman bit



8 bits



4 bits



2 bits



1 bit

- 8 bit: gradasi masih terlihat halus
- 4 bit: mulai tampak pita warna pada wajah
- 2 bit: bentuk masih dikenali, gradasi hilang
- 1 bit: *tinggal hitam dan putih*
Di sinilah dithering dibutuhkan



Tiga cara memangkas jumlah warna

Warna terdekat

Tiap piksel dibulatkan langsung ke warna palet terdekat.

Banding dan detail hilang

Ordered dithering

Piksel dibandingkan dengan matriks ambang tetap (Bayer).

Cepat, pola kotak terlihat

Error diffusion

Error pembulatan disebar ke tetangga yang belum diproses.

Paling halus, tepi ikut kuat

Penyederhanaan warna menjadi delapan warna

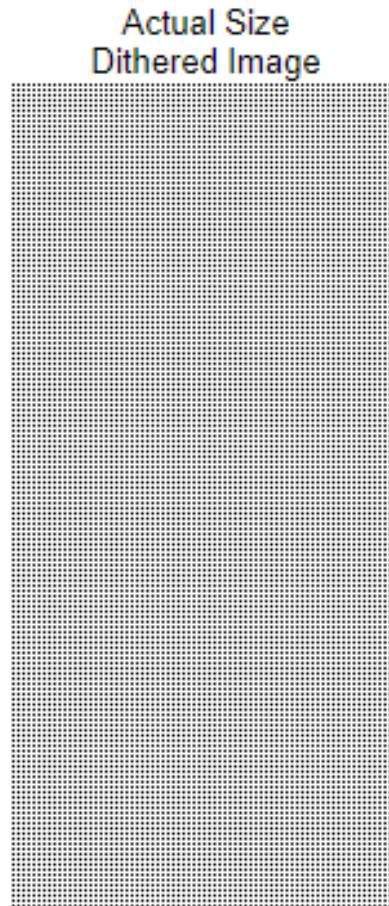
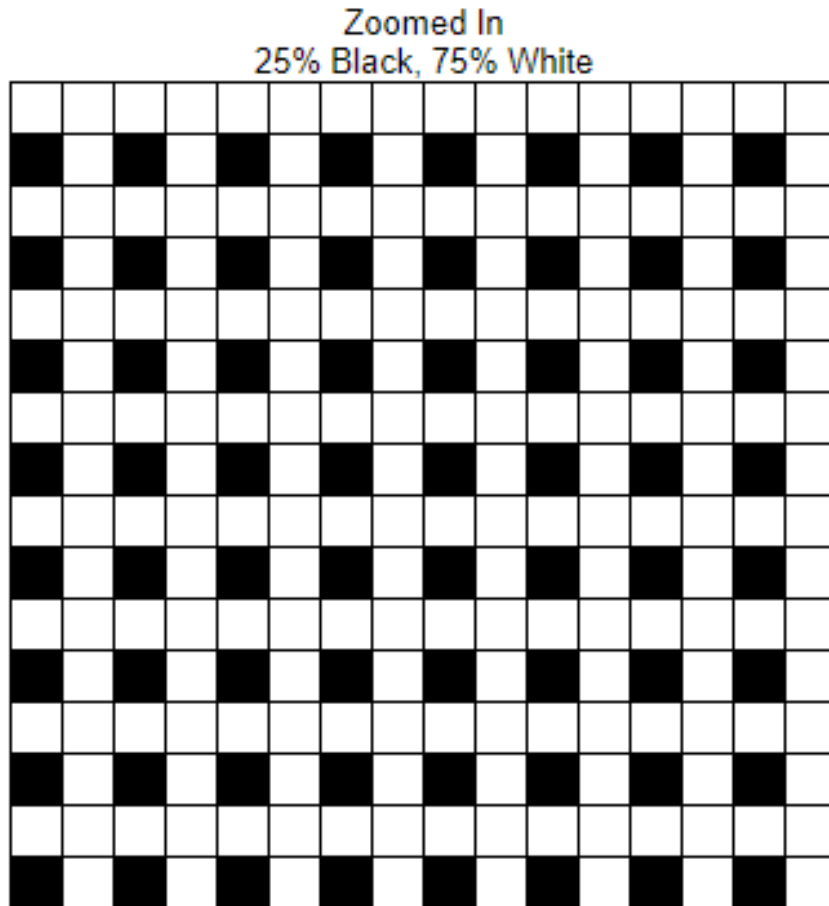


Contoh : Penyederhanaan Gambar Menjadi 8 Warna
Memetakan 16 Juta warna RGB ke dalam 8 warna saja yaitu hitam, merah, hijau, kuning, biru, cyan, magenta, putih

- TANPA ERROR DIFFUSION
- Tiap piksel langsung diganti warna ke palet terdekat
- Hasilnya banyak bagian berwarna sama
- Gradasi pada wajah dan latar hilang

Bandingkan dengan hasil dithering pada slide berikutnya

Ilusi abu-abu dari titik hitam dan putih

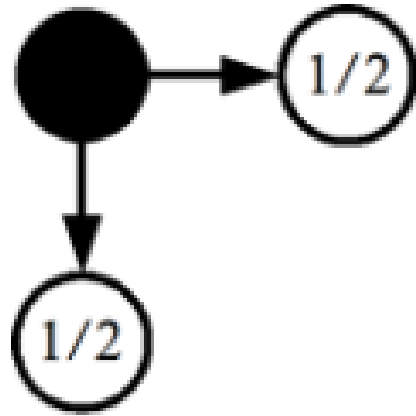


- MENGAPA BISA TERLIHAT ABU-ABU?
- Kiri: pola diperbesar, isinya hanya hitam dan putih
- Kanan: pola yang sama pada ukuran sebenarnya
- Mata merata-ratakan titik yang terlalu kecil untuk dipisahkan

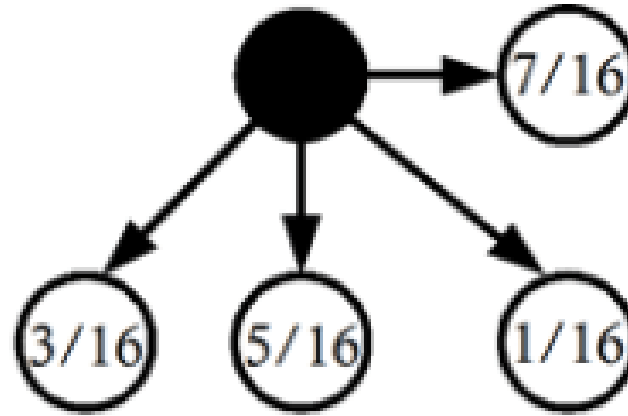
Seperempat titik hitam terbaca sebagai abu-abu terang

Representasi ilusi cahaya abu-abu dengan menggunakan warna hitam putih.

Diagram node error diffusion

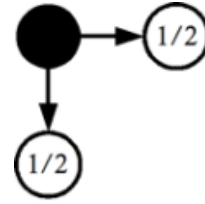


Bentuk sederhana: error dibagi dua



Floyd - Steinberg: $7/16$, $3/16$, $5/16$, dan $1/16$

Error diffusion



error = nilai asli - warna terdekat lalu error dibagi ke tetangga

TAHAPAN

- Ambil piksel yang sedang diproses
- Bulatkan ke warna palet terdekat, tulis sebagai keluaran
- Hitung error, yaitu selisih nilai asli dengan warna terdekat
- Bagikan sebagian error ke tetangga yang belum diproses
- Lanjut ke piksel berikutnya, error ikut terbawa

ATURAN YANG TIDAK BOLEH DILANGGAR

- Jumlah seluruh bobot harus sama dengan 1
- Error hanya boleh dibagikan ke piksel yang belum diproses
- Nilai tetangga tetap dijaga pada rentang 0 sampai 255
- Untuk citra warna, ulangi terpisah pada kanal R, G, dan B
- Simpan citra kerja sebagai float agar error tidak terpotong

Karena hasil satu piksel memengaruhi piksel lain, error diffusion tergolong area operation.

Error diffusion: contoh satu piksel

Palet hanya hitam (0) dan putih (255)

CONTOH SATU PIKSEL

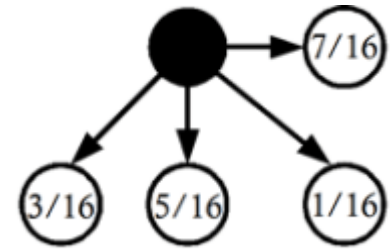
1. Piksel (x, y) bernilai 150
2. Warna terdekat = 255, itu keluarannya
3. $\text{error} = 150 - 255 = -105$
4. Kanan: $200 + 0,5(-105) = 147,5$
5. Bawah: $30 + 0,5(-105) = -22,5$, truncate 0

PSEUDOCODE

```
error = warnaAsli - warnaTerdekat
SetPixel(x+1, y) =
    Truncate(GetPixel(x+1, y) + 0.5 * error)
SetPixel(x, y+1) =
    Truncate(GetPixel(x, y+1) + 0.5 * error)
```

Truncate menjaga nilai tetangga tetap berada di rentang 0 sampai 255.

Kernel Floyd – Steinberg



Pemindaian dari kiri ke kanan, baris atas ke bawah. Tanda # adalah piksel yang sedang diproses.

BENTUK KERNEL (dibagi 16)

$$\frac{1}{16} \begin{bmatrix} - & \# & 7 \\ 3 & 5 & 1 \end{bmatrix}$$

- Kanan: $7/16$ · Bawah: $5/16$
- *Bawah kiri: $3/16$ · Bawah kanan: $1/16$*

PSEUDOCODE (x = kolom)

1. error = warnaAsli - warnaTerdekat
 2. $I(x+1, y) \quad += \quad 7/16 * \text{error}$
 3. $I(x-1, y+1) \quad += \quad 3/16 * \text{error}$
 4. $I(x, y+1) \quad += \quad 5/16 * \text{error}$
 5. $I(x+1, y+1) \quad += \quad 1/16 * \text{error}$
- Jumlah bobot: $7 + 5 + 3 + 1 = 16$, jadi tepat 1*

Tuliskan konvensi sumbu Anda lebih dulu. Bila x dipakai sebagai baris, seluruh rumus ikut berpindah.

Contoh hitung satu piksel

Palet 2 warna: 0 dan 255. Piksel yang diproses bernilai 130.

LANGKAH

- Jarak ke 0 adalah 130, jarak ke 255 adalah 125
- Warna terdekat: 255, jadi keluaran piksel ini putih
- $\text{error} = 130 - 255 = -125$
- Error negatif berarti keluaran terlalu terang

PEMBAGIAN ERROR

- Kanan : $7/16 \times (-125) = -54,7$
- Bawah kiri : $3/16 \times (-125) = -23,4$
- Bawah : $5/16 \times (-125) = -39,1$
- Bawah kanan : $1/16 \times (-125) = -7,8$

Jumlahnya kembali -125

Bila tetangga kanan semula 140, nilainya menjadi 85,3 sehingga dibulatkan ke 0 alias hitam.

Contoh hitung satu piksel

Tetangga mengumpulkan error dulu, pembulatan menunggu gilirannya.

CONTOH PIXEL TETANGGA KANAN

1. Nilai awalnya 140
2. Terima -54,7 dari kiri, jadi 85,3
3. Terima +12,3 dari atas, jadi 97,6
4. Gilirannya: 97,6 dibulatkan ke 0
5. error barunya $97,6 - 0 = +97,6$

Piksel yang sudah jadi keluaran tidak pernah diubah lagi oleh error berikutnya.

Penyebaran error pada matriks 3 x 3

Angka di kotak adalah nilai pixel awal, belum warna keluaran.

Sebelum: piksel biru sedang diproses

	130	140
90	200	60

Sesudah: error -125 sudah dibagikan

	255	85,3
66,6	160,9	52,2

Bobot: kanan 7/16, bawah 5/16, bawah kiri 3/16, bawah kanan 1/16.

Empat tetangga berubah nilainya, tiga kotak sisanya menunggu giliran masing-masing.

Posisi penerapan rumus pada citra 3 x 3

X	R1	
R3	R4	

-	-	-
X	R1	
R3	R4	

-	-	-
-	-	-
X	R1	

-	X	R1
R2	R3	R4

-	-	-
-	X	R1
R2	R3	R4

-	-	-
-	-	-
-	X	R1

-	-	X
	R2	R3

-	-	-
-	-	X
	R2	R3

-	-	-
-	-	-
-	-	-

- Kotak biru: piksel yang diproses

Bobot Floyd–Steinberg

R1 kanan, 7/16

R2 bawah kiri, 3/16

R3 bawah, 5/16

R4 bawah kanan, 1/16

Jarvis, Judice & Ninke

Kernel 5 x 3 dengan pembagi 48, disebut minimized average error

BENTUK

$$\frac{1}{48} \begin{bmatrix} - & - & \# & 7 & 5 \\ 3 & 5 & 7 & 5 & 3 \\ 1 & 3 & 5 & 3 & 1 \end{bmatrix}$$

Error disebar sampai dua baris dan dua kolom ke depan

DIBANDING FLOYD – STEINBERG

- Sebaran error lebih luas, pola titik terlihat lebih halus
- Tepi objek sedikit lebih lembut
- Perhitungan lebih berat karena 12 tetangga
- Butuh penyangga dua baris, bukan satu

Floyd – Steinberg tetap yang paling sering dipakai

Kernel lain yang sering disebut: Stucki, Burkes, Sierra, dan Atkinson.



Enam kekeliruan yang paling sering terjadi

1 Sumbu
Baris ditulis x

2 Tipe data
Error terpotong

3 Clipping
Nilai keluar rentang

4 Tepi citra
Tetangga di luar

5 Urutan
Dithering lebih dulu

6 Kanal RGB
Error antar kanal



Membandingkan hasil akhir

Thresholding

Setiap piksel dibandingkan dengan satu nilai ambang tetap.

Gradasi hilang sama sekali

Ordered dithering

Ambang berubah menurut posisi mengikuti matriks Bayer.

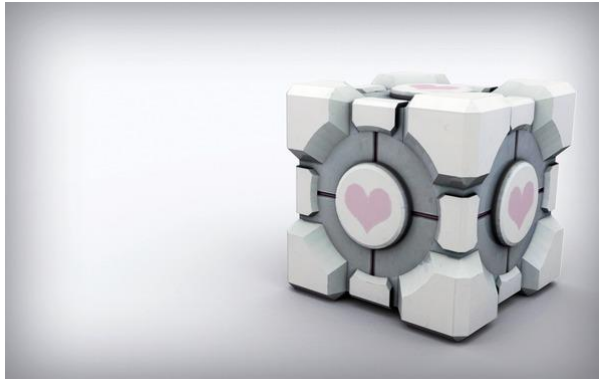
Gradasi kembali, pola teratur

Floyd - Steinberg

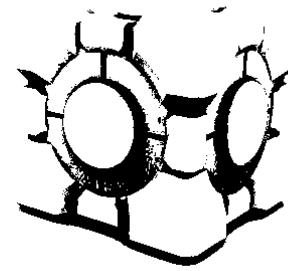
Error disebar ke tetangga sehingga pola titik tampak acak.

Paling halus, teks tetap terbaca

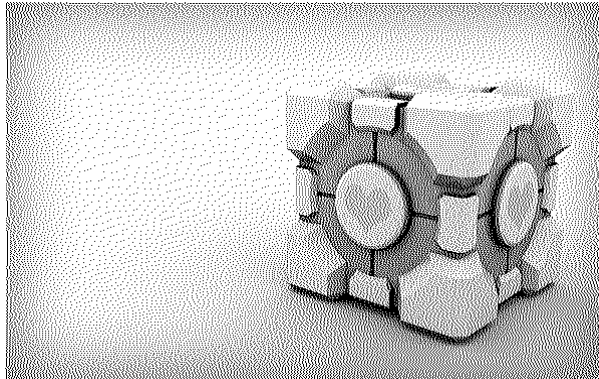
Contoh hasil dithering



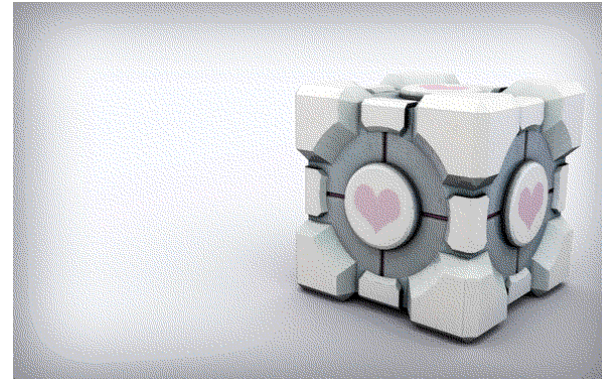
Citra asli



Cetak hitam putih tanpa dithering



Dithering hitam putih



Dithering berwarna

Referensi

Bacaan untuk mendalami materi pertemuan ini

BUKU DAN CATATAN KULIAH

- Gonzalez & Woods, Digital Image Processing, bab 3
- Rinaldi Munir, Pengolahan Citra Digital, bab histogram
- Dokumentasi OpenCV: Histograms dan CLAHE
- Dokumentasi NumPy: `numpy.histogram`

SUMBER DITHERING

- Floyd & Steinberg (1976), An adaptive algorithm for spatial greyscale
- Jarvis, Judice & Ninke (1976), A survey of techniques for display of
- continuous tone pictures on bilevel displays
- Tanner Helland, Dithering: eleven algorithms and source code
- *Ulichney, Digital Halftoning, MIT Press*



Terima kasih!

Team Teaching PCVK

